QUANT_PIPELINE

WorldQuant BRAIN Alpha Research Blueprint

A practical, research-first guide to building an AI-assisted system that proposes, tests, ranks, documents and prepares candidate alphas for WorldQuant BRAIN.

Primary objective	Discover robust and meaningfully different alpha candidates; validate them in the environment where they will be evaluated.
Core principle	AI proposes hypotheses. Deterministic code checks them. Statistical validation challenges them. BRAIN evaluates platform-specific performance.
Recommended stack	Python research engine, PostgreSQL experiment store, existing LLM, optional React/Node.js dashboard.
Operating mode	Hybrid: use external data for independent exploration when helpful, and BRAIN datasets/simulations as the final authority for BRAIN candidates.

Prepared: 22 October 2026 | Version 1.0

This is a technical research plan, not a promise of profitability, acceptance, or compensation.

Contents

1. Executive summary and success criteria
2. What WorldQuant BRAIN is—and what it is not
3. System architecture
4. Research workflow from hypothesis to submission
5. Data strategy and governance
6. Alpha discovery: how to use AI well
7. Validation, statistics and overfitting controls
8. BRAIN integration and submission workflow
9. Database design and experiment tracking
10. Recommended repository structure
11. Implementation roadmap and milestones
12. Quality gates, metrics and operating checklist
13. Risks, security and responsible automation
14. Learning plan and recommended resources
15. Example research cycle

Appendix A. Candidate alpha record

Appendix B. Definition of done

Sources and important caveats

How to use this document

Follow the phases in order. Do not begin with autonomous bulk submission. First build a small, inspectable system that can reproduce a baseline experiment, explain why a candidate exists, and preserve both positive and negative results.

1. Executive summary and success criteria

Quant_Pipeline should be a quantitative research workbench—not a black box that emits formulas. Its purpose is to shorten the research cycle while preserving scientific discipline: a hypothesis is proposed, translated into an expression, checked, tested, compared against baselines, recorded, and only then considered for BRAIN submission.

The recommended first version is deliberately narrow:

- One research domain at a time, initially simple daily equity signals.
- Use BRAIN's own fields and simulation interface for platform-aligned work.
- Use external historical data only where it adds independent exploratory value; do not assume different datasets are interchangeable.
- Generate small batches of distinct candidates instead of thousands of superficial variations.
- Store every experiment, including failed and rejected candidates.
- Keep a human approval step before any final submission or account-affecting action.

What success looks like

Milestone	Evidence of completion
Reproducible baseline	You can recreate a manually researched candidate and its configuration from saved records.
Valid expression pipeline	Expressions pass local structural checks and platform validation before consuming simulation effort.
Research traceability	Every candidate links to a hypothesis, data fields, transformations, run settings, results and decision.
Robust evaluation	Selection uses out-of-sample evidence, sensitivity checks and explicit overfitting safeguards.
Controlled BRAIN workflow	Only permitted, account-authorized workflows are used; submission decisions remain auditable.

Important: no pipeline can guarantee a 'perfect' alpha or predictable earnings. Markets change, apparent patterns can be statistical noise, and platform criteria or account features can change. Aim for a robust research process, not a guaranteed result.

2. What WorldQuant BRAIN is—and what it is not

WorldQuant describes BRAIN as a simulation platform with datasets, tools and performance dashboards for developing and testing alphas. Its public material describes alpha research and a pathway through which successful participants may become research consultants. Advanced features—including Python API use—are described as consultant-program benefits, so access should be verified inside your account rather than assumed. [1][2]

Design implications

- BRAIN is the authoritative environment for evaluating a candidate intended for BRAIN; an independent backtest is supporting evidence, not a substitute.
- Data-field availability, regions, simulation settings, operator syntax and account features may depend on the user's access level.
- Do not scrape private endpoints, bypass platform controls, share credentials, or automate actions contrary to current terms or program rules.
- Build an adapter boundary around BRAIN integration. If your account lacks API access, the rest of the research system should still work and prepare expressions for manual use.

- Keep research eligibility, consultant onboarding and compensation separate from technical alpha quality. None is guaranteed.

Public links to check

WorldQuant BRAIN: https://www.worldquant.com/brain/ | BRAIN platform: https://platform.worldquantbrain.com/ | Consultant Program: https://platform.worldquantbrain.com/consultant-program/

The official consultant-program page lists Kenya among supported regions, but eligibility, onboarding, account standing and compensation conditions still apply. Competition eligibility and prize rules can be different from consultant-program rules; read the relevant rules separately. [2][3]

3. System architecture

Build a pipeline of small, testable modules. The first release can run as one Python application; split services only when a real workload or reliability requirement justifies it.

Logical flow

Layer	Responsibilities	Output
1. Research configuration	Universe, region, delay, neutralization, truncation, decay, data fields and date windows.	Versioned run configuration
2. Data catalog and quality	Record data source, field definition, timestamps, missingness, coverage and permitted use.	Trusted dataset/field metadata
3. Hypothesis engine	Create a falsifiable economic or behavioral explanation and expected sign/horizon.	Hypothesis record
4. Candidate generator	Turn hypotheses into expressions using allowed operators and fields.	Candidate expressions
5. Static validator	Check syntax tree, operator arity, field names, type compatibility, complexity and duplicates.	Valid candidates or clear errors
6. Evaluation engine	Run local diagnostics and permitted BRAIN simulations; collect results.	Metrics and diagnostics
7. Selection and portfolio diversity	Rank robust candidates; penalize complexity, instability and similarity.	Shortlist
8. Human review and submission	Review evidence, verify platform settings and approve final action.	Submission record
9. Research memory	Preserve results, failure reasons, versions and follow-up experiments.	Queryable research history

Suggested initial stack

Area	Recommendation	Why
Research	Python; pandas, NumPy, SciPy, statsmodels; scikit-learn when needed	Strong ecosystem for time series, statistics and experimentation.
Data storage	PostgreSQL; Parquet files for larger local research tables	SQL for metadata/results; columnar files for efficient research datasets.
LLM	An existing model behind a provider-neutral interface	Avoid building/training a model before the research loop proves useful.
Dashboard	Optional React + Node/Express	Useful later for monitoring and review; not required for MVP.
Scheduling	Simple CLI / task runner first	Less operational complexity; add a queue only when needed.

Do not start with microservices, GPUs, model fine-tuning or an elaborate agent framework. The hard problem is research validity, not infrastructure scale.

4. Research workflow: from hypothesis to submission

Step 1 — Ask a falsifiable question

Example: 'Do stocks with unusually strong recent returns relative to their own recent history tend to continue outperforming over the next short horizon?' Define the universe, holding horizon, expected direction and plausible mechanism.

Step 2 — Identify data and timing

Choose fields available in BRAIN. Document what each field measures and when it becomes known. Do not silently substitute an external field with different semantics.

Step 3 — Generate a small candidate family

Generate a limited set of expressions that vary for a research reason: lookback, normalization, ranking, group treatment or smoothing. Each variation should test a clear idea.

Step 4 — Run deterministic checks

Reject unknown fields/operators, invalid parameter ranges, unsupported syntax, excessive expression depth, divide-by-zero risks where applicable, and duplicate candidates.

Step 5 — Run inexpensive diagnostics

Check missingness, coverage, outliers, feature distributions, basic correlations and temporal stability where local data permits. These are screening tools, not proof.

Step 6 — Simulate in BRAIN

Use account-authorized simulation features and record all configuration settings. Avoid changing several settings at once unless running a planned experiment.

Step 7 — Evaluate beyond one headline score

Review the platform's available performance and robustness metrics, turnover, drawdown/risk behavior, subperiod stability, universe sensitivity and correlation with existing candidates.

Step 8 — Challenge the result

Ask whether the candidate is a data leak, a duplicate, an unstable parameter fit, a sector bet, a microcap/liquidity artifact or a result selected from too many trials.

Step 9 — Decide and record

Mark as reject, revise, hold for more evidence or submit for review. Record the reason. Do not delete failures.

Step 10 — Learn from the experiment

Use the result to refine research priorities, not to blindly prompt the LLM to produce near-identical variations.

5. Data strategy and governance

Start with the fields available in your own BRAIN account. Create a data catalog before building an external market-data warehouse. A large dataset is not automatically useful; correct definitions and point-in-time availability matter more than volume.

For every field or external series, record

- Source, provider, license/terms and permitted use.
- Field name, units, frequency, currency, adjustment method and economic interpretation.
- Observation timestamp, publication timestamp and when the value would realistically be available to a strategy.
- Coverage by asset, region and time; missing values; stale observations; known revisions.
- Corporate-action treatment, delisted names and universe membership where relevant.
- Refresh cadence, data version and quality checks.

External data: use it selectively

External sources can help explore ideas or independently test a concept, but the first version should not depend on an expensive real-time feed. For daily equity research, historical observations with defensible timestamps are often enough to start. Confirm API terms, redistribution restrictions, rate limits and data licensing before storing or using third-party data.

Recommended data-quality checks

Check	Question	Action on failure
Timestamp integrity	Was each input available at the simulated decision time?	Stop run; repair timing logic.
Missingness	Are missing values clustered by date, asset or field?	Document and apply a defensible policy; don't fill blindly.
Coverage	Does coverage change sharply through time?	Inspect universe/data availability bias.
Duplicates	Are timestamps or records duplicated?	Deduplicate with explicit precedence rules.
Outliers	Are extreme values real or data errors?	Investigate before winsorization or clipping.
Revisions	Were later revisions used as if known earlier?	Use point-in-time values or exclude the field.

For BRAIN work, the platform's field definitions and simulation data should take precedence over an external proxy when judging the exact BRAIN alpha. Never assume an external data download can be uploaded or used in BRAIN unless the platform explicitly supports that workflow.

6. Alpha discovery: how to use AI well

Use AI as a hypothesis and coding assistant—not as an oracle. Language models can propose plausible stories, produce invalid expressions, repeat known ideas or confidently misinterpret statistics. Every generated item must be machine-checkable and independently evaluated.

Recommended AI stages

Stage	Input	Expected output	Guardrail
Hypothesis	Research domain, field catalog, known literature notes	Falsifiable hypothesis and mechanism	Require expected direction, horizon and disconfirming test.
Expression design	Allowed operators, field types, configuration	Small family of candidate expressions	Only whitelisted operators/fields; parse, don't trust prose.
Critique	Candidate and hypothesis	Potential leakage, redundancy and failure modes	Critique must not replace empirical checks.
Result analysis	Metrics and experiment history	Evidence summary and next experiment	Provide actual metrics; never let model invent missing values.
Research memory	Past candidate records	Related experiments and prior failures	Retrieve from database; cite experiment IDs.

Use structured outputs

A hypothesis response should include: hypothesis_id; economic rationale; field candidates; expected sign; holding horizon; proposed expression family; assumptions; likely confounders; falsification tests; and novelty notes. The application should reject incomplete responses.

Example prompt template

```
You are assisting a quantitative researcher. Propose at most 5 distinct, falsifiable hypotheses for the selected research domain. Use only fields and operators from the supplied catalog. For each hypothesis, provide: mechanism, expected direction, horizon, expression candidates, assumptions, leakage risks, confounders, a falsification test, and why it is different from the prior experiments supplied. Do not claim performance without measured results. Return structured JSON matching the supplied schema.
```

Avoid optimizing only the visible score

If the LLM sees every validation result and keeps generating variants against the same holdout period, that holdout gradually becomes part of the training process. Keep a final untouched evaluation window, limit experiment budgets, log every attempt and prefer robust families over one lucky formula.

7. Validation, statistics and overfitting controls

This section is the heart of the pipeline. A candidate can have an attractive simulation score and still be fragile, redundant or produced by chance. Use multiple layers of evidence and respect the metrics actually provided by BRAIN.

Minimum validation layers

- Syntax and data checks: valid fields/operators, expected types, adequate coverage and no accidental future information.
- Baseline comparison: compare against simple, interpretable baselines and a no-signal/null benchmark where feasible.
- Time separation: develop on earlier periods, validate on later periods, and preserve a final untouched period for independent assessment when your data workflow supports it.
- Walk-forward tests: repeat training/selection on rolling historical windows rather than relying on one split.
- Parameter sensitivity: nearby lookbacks or settings should not cause the signal to collapse without a plausible reason.
- Subperiod and universe checks: inspect performance across different periods, market conditions and relevant subsets where supported.
- Turnover and costs: consider whether the signal would remain meaningful after plausible transaction costs and turnover; do not assume gross results equal net results.
- Correlation and redundancy: assess similarity to your own candidate library and use platform-provided correlation or value-add measures where available.
- Multiple testing: log every experiment and treat the best result from a large search as more suspicious than a single pre-specified test.
- Reproducibility: save configuration, code version, data version, timestamps and results so the experiment can be repeated.

Useful statistical concepts

Concept	Plain-language meaning	Use in the pipeline
Out-of-sample (OOS)	Data not used to design or tune the candidate.	Check whether the pattern generalizes.
Look-ahead bias	Accidentally using information from the future.	Timestamp and feature-availability checks.
Survivorship bias	Only studying assets that survived or remain listed.	Use point-in-time universes where possible.
Multiple testing	Trying many ideas increases the chance of a lucky result.	Track trials, constrain search, adjust interpretation.
Turnover	How frequently portfolio positions change.	Assess trading friction and signal stability.
Correlation	How similarly two signals behave.	Avoid submitting a portfolio of near-duplicates.
Regime sensitivity	A signal may work only in certain market conditions.	Inspect subperiods; avoid overclaiming universality.

Metrics policy

Do not hard-code universal pass thresholds copied from a blog. BRAIN's required tests, score definitions and thresholds may vary by settings or program. Store the metrics the platform actually reports, record the date and context, and make configurable gates. Use platform rules for submission eligibility and independent checks for research quality.

Candidate rejection examples

- Fails platform syntax or required simulation tests.
- Depends on a field whose publication time is later than the decision time.
- Performance vanishes in later periods or nearby parameter settings.

- Nearly duplicates a better candidate without a clear diversification benefit.
- Only works in a tiny, unstable subset or appears driven by a data defect.
- Results cannot be reproduced because configuration or data provenance is missing.

8. BRAIN integration and submission workflow

Treat integration as a replaceable adapter. This lets your research engine remain useful even if API access is unavailable or platform interfaces change.

Three operating modes

Mode	How it works	When to use it
Manual-first	Pipeline prepares expression, hypothesis and review checklist; you paste and simulate in the platform.	Best starting point for a new or basic account.
Assisted	Authorized integration submits simulation requests or retrieves results with user review, if permitted by account features.	After manual workflow is understood and access is verified.
Automated batch	Runs bounded experiment batches, polls results and records outcomes through documented, permitted mechanisms.	Only after API access, terms, rate limits and safeguards are confirmed.

Integration requirements

- Verify access and current official documentation from inside the account before implementation.
- Use only documented/authorized endpoints and authentication methods. Never hard-code credentials or store passwords in source control.
- Respect rate limits, retry policies, quotas and platform restrictions. Implement exponential backoff for transient errors and stop on authentication/permission errors.
- Make jobs idempotent: retries must not silently create duplicate submissions or lose the link between candidate and simulation.
- Keep a dry-run mode that prints the planned action without contacting the platform.
- Require explicit approval before final submission until you have tested the workflow thoroughly and confirmed automation is permitted.
- Store request metadata and platform response identifiers, but avoid logging secrets or unnecessary personal data.

Important boundary

The public WorldQuant consultant-program page advertises Python API access as an advanced consultant feature. Therefore, do not design the MVP assuming every BRAIN user can programmatically submit or retrieve simulations. Build the adapter interface first and implement the actual connector only against permitted, current documentation. [2]

9. Database design and experiment tracking

Your research memory is a competitive advantage. It prevents repeating failed work, makes results auditable and lets the AI compare a new idea against actual prior experiments.

Suggested tables

Table	Key fields	Purpose
research_hypotheses	id, title, rationale, expected_sign, horizon, created_at, status	Store the scientific question and reasoning.
data_fields	field_key, source, description, type, timing, region, permitted_use	Catalog field semantics and restrictions.
alpha_candidates	id, hypothesis_id, expression, normalized_expression, complexity, version, status	Store every candidate and its lifecycle.
experiments	id, candidate_id, config_json, code_version, started_at, finished_at, status	Record the exact run and settings.
experiment_metrics	experiment_id, metric_name, metric_value, source, context_json	Store platform/local metrics without assuming all runs share metrics.
decisions	candidate_id, decision, reason, reviewer, created_at	Explain accept/reject/revise decisions.
submissions	candidate_id, platform_id, submitted_at, state, response_metadata	Track submission state and platform identifiers.
model_runs	provider, model, prompt_version, output_hash, cost_estimate, timestamp	Audit AI output and changes over time.

Research statuses

```
PROPOSED → STATIC_CHECKED → DIAGNOSTICS_PASSED → SIMULATION_QUEUED → SIMULATED → REVIEW_REQUIRED → APPROVED_FOR_SUBMISSION / REJECTED / NEEDS_REVISION → SUBMITTED (if approved) → PLATFORM_RESULT_RECORDED.
```

Never overwrite an old expression or result when making a change. Create a new version linked to its parent candidate. That preserves the research history and reduces accidental cherry-picking.

10. Recommended repository structure

Start as a modular Python project. The dashboard and Node.js API can be added later without changing the research core.

```
quant-pipeline/ README.md pyproject.toml .env.example configs/ research.yaml data_sources.yaml docs/ architecture.md research_protocol.md decisions.md src/quant_pipeline/ config/ schemas.py data/ catalog.py quality.py loaders.py hypotheses/ schema.py generator.py critique.py expressions/ parser.py validator.py canonicalize.py evaluation/ diagnostics.py validation.py ranking.py correlation.py brain/ adapter.py manual_export.py authorized_client.py storage/ database.py repositories.py orchestration/ runner.py policies.py reporting/ report.py tests/ unit/ integration/ fixtures/ experiments/ notebooks/ artifacts/ reports/ # generated, not secrets scripts/ run_research.py export_candidates.py
```

Implementation rules

- Use type hints, schema validation and unit tests around expression parsing and configuration.
- Keep secrets in environment variables or a secret manager; commit only .env.example with placeholder names.
- Pin dependency versions and record the Python version.
- Separate pure functions (expression validation, metrics transforms) from network calls.
- Write tests for failure paths, retries, duplicate candidates and missing metrics—not only the happy path.
- Keep notebooks for exploration, but move repeatable research logic into tested modules.

11. Implementation roadmap and milestones

The timeline below is a sequence of milestones, not a guarantee of calendar completion. Move forward only when the exit criteria for each phase are satisfied.

Phase	Build	Exit criteria
0. Platform reconnaissance	Create/verify account, learn expression language, identify fields, operators, simulation settings and access level.	A written capability matrix and 3–5 manually understood baseline experiments.
1. Research notebook	Implement a hypothesis template, experiment log and candidate registry.	Every baseline has its rationale, expression, configuration, result and decision recorded.
2. Expression engine	Build parser/validator, canonicalization, field/operator whitelist and complexity limits.	Unit tests cover valid expressions, invalid operators, unknown fields and duplicates.
3. AI-assisted discovery	Add structured hypothesis generation and expression proposals.	Outputs are schema-valid, reproducible and reviewable; no unverified claims are accepted.
4. Evaluation and research memory	Store simulation metrics; implement ranking, robustness notes and similarity tracking.	Can compare candidates and explain why each was kept/rejected.
5. BRAIN adapter	Begin with manual export; add authorized API connector only if access permits.	Dry run, authentication handling, retry tests, audit logs and approval gate work.
6. Controlled automation	Bounded batch scheduling, quotas, monitoring and stop conditions.	No duplicate action on retry; easy kill switch; human review remains available.
7. Advanced research	Walk-forward studies, stronger multiple-testing controls, candidate-family analysis.	Research protocol is documented and final holdout remains protected.

Suggested first two weeks

- Days 1–2: learn the BRAIN workflow, field definitions, operator docs and current account features.
- Days 3–4: research and record a few simple baseline candidates manually.
- Days 5–7: create the Python repository, configuration schema, database and candidate record format.
- Days 8–10: implement expression validation and tests; do not automate submission yet.
- Days 11–14: add LLM hypothesis generation, review prompts and comparison against prior experiments.

12. Quality gates, metrics and operating checklist

Use explicit gates. A candidate should not advance simply because the AI sounds confident or one metric looks good.

Gate	Pass condition	If it fails
G0: Provenance	Hypothesis, field definitions, configuration and versions recorded.	Return to research record; do not simulate yet.
G1: Syntax	Expression parses and uses permitted fields/operators.	Reject or revise.
G2: Data integrity	No known future leakage or unexplained quality defects.	Stop and investigate.
G3: Baseline value	Candidate offers a reasoned test beyond a duplicate or trivial variant.	Deprioritize or redesign hypothesis.
G4: Platform simulation	Run completed with settings and metrics captured.	Record error; do not invent missing results.
G5: Robustness	Available out-of-sample, stability and sensitivity checks are acceptable for the research policy.	Reject, revise or gather evidence.
G6: Diversity	Not redundant with existing candidates, or adds a defensible diversification benefit.	Keep only if incremental value is evidenced.

Gate	Pass condition	If it fails
G7: Approval	Human reviewer approves the exact expression and intended action.	No submission.

Weekly research review

- How many hypotheses were genuinely distinct, not merely how many formulas were generated?
- Which data fields or assumptions caused the most failed experiments?
- Did any candidates pass only because the same validation window was repeatedly reused?
- Are the best candidates too correlated with each other or with known baseline signals?
- Which failures should change future search priorities?
- Are API use, data licensing, credentials and platform rules still compliant?

13. Risks, security and responsible automation

Risk	Mitigation
Overfitting / selection bias	Limit search budgets, log all trials, protect final holdout, use walk-forward validation and require robustness evidence.
Look-ahead / survivorship bias	Point-in-time fields/universes where available; explicit timestamp policy; audit feature availability.
LLM hallucination	Treat outputs as untrusted proposals; validate against schemas and actual platform docs; never invent metrics.
Credential leakage	Environment variables or secrets manager; redact logs; least privilege; rotate exposed secrets.
Unauthorized automation	Use only permitted documented methods; verify current terms and account permissions; start manual-first.
Runaway cost / request volume	Per-run and daily budgets, concurrency limits, backoff, circuit breaker and kill switch.
Duplicate or repeated submissions	Idempotency keys where supported, candidate hashes, confirmation steps and durable state transitions.
Data license violation	Record terms and allowed use; don't redistribute proprietary or restricted datasets.
False income expectations	Treat compensation as conditional; separate research milestones from earnings assumptions.

Mandatory safety controls before any automated batch

- Dry-run mode enabled by default.
- Maximum candidates and maximum requests per batch configured.
- Explicit daily budget and stop conditions.
- Authentication and permission errors halt the run instead of being retried indefinitely.
- Every candidate is linked to one unique experiment and configuration.
- Human approval required for final submission unless a later documented policy explicitly permits a different controlled process.
- Emergency stop works even if a worker is running.

14. Learning plan and recommended resources

Build the project while learning the concepts. You do not need to master every area before starting, but you should understand the statistics behind the decisions your pipeline makes.

Topic	Learn	Apply it in Quant_Pipeline
Python research	pandas, NumPy, plotting, tests, type hints, virtual environments	Data quality, expression tooling and reproducible experiments.
Probability and statistics	Distributions, sampling, hypothesis tests, confidence intervals, correlation	Understand uncertainty and avoid overinterpreting noisy metrics.
Time series	Returns, lags, rolling windows, autocorrelation, stationarity limitations	Correct time alignment and temporal validation.
Portfolio and market basics	Momentum, reversal, volatility, liquidity, neutralization, turnover	Develop interpretable signal families and identify confounders.
Experimental design	Train/validation/test separation, walk-forward testing, multiple testing	Prevent the research loop from selecting lucky candidates.

Topic	Learn	Apply it in Quant_Pipeline
Software engineering	SQL, migrations, API boundaries, logging, unit/integration tests	Reliable records and safe platform integration.

Official starting points

• WorldQuant BRAIN overview: https://www.worldquant.com/brain/ [1]
• BRAIN Consultant Program: https://platform.worldquantbrain.com/consultant-program/ [2]
• Learn2Quant educational series: https://www.worldquant.com/learn2quant/ [4]
• BRAIN platform: https://platform.worldquantbrain.com/

Start with the platform's own learning materials and field/operator documentation. Use third-party tutorials as supplementary explanations, not as the authority for current platform syntax or rules.

15. Example research cycle

This is a workflow illustration, not a claim that the expression is profitable or accepted. Confirm all fields, operators and syntax in your own BRAIN environment before use.

Hypothesis

Recent relative strength may contain short-horizon continuation information. A competing explanation is that the signal is simply exposure to sector, size, volatility or a recent market regime.

Research protocol

- Choose a supported return/price-derived field or build the idea using supported BRAIN operators and fields.
- Specify the intended horizon and the rationale before seeing results.
- Generate a small family of candidates with distinct, documented lookback or normalization choices.
- Validate field names and operator arity using the platform's current language reference.
- Run simulations with fixed settings and record each result, including failures.
- Compare available performance measures, turnover, stability and correlation to the existing candidate library.
- Check whether the candidate adds information beyond simple baselines and whether it survives an independent time period.
- Decide: reject, revise, gather more evidence or submit for human review.

Illustrative pseudocode

```
hypothesis = create_hypothesis( question="Does recent relative strength predict future returns?", expected_direction="positive", horizon="predefined by research protocol", confounders=["sector exposure", "volatility", "regime"] ) candidates = llm.propose_candidates( hypothesis=hypothesis, allowed_fields=brain_field_catalog, allowed_operators=brain_operator_catalog, max_candidates=5 ) for candidate in candidates: validate_schema(candidate) validate_expression(candidate.expression) if candidate_is_duplicate(candidate, experiment_store): record_rejection(candidate, "duplicate") continue local_diagnostics = run_data_and_leakage_checks(candidate) if not local_diagnostics.passed: record_rejection(candidate, local_diagnostics.reason) continue result = run_authorized_brain_workflow(candidate) # manual adapter if no API save_experiment(candidate, result, config, code_version) decision = review_robustness_and_diversity(result) record_decision(candidate, decision)
```

The key learning loop

A weak result should update the research record. For example, if candidates repeatedly fail after sector controls or show unstable performance, future hypotheses should explicitly address those weaknesses. Do not respond by asking the AI to generate hundreds more variations of the same failed idea.

Appendix A. Candidate alpha record

Use this template for every candidate, including rejected ones.

Field	What to record
Candidate ID / version	Stable unique ID, parent candidate ID and version number.
Hypothesis	Question, rationale, expected sign, intended horizon and disconfirming test.
Expression	Exact expression string and canonicalized representation.
Data provenance	Fields, source, region, timing assumptions and relevant definitions.
Configuration	All applicable simulation settings, universe, delay and neutralization parameters.
Experiment metadata	Run ID, timestamp, code version, prompt version and data version.
Results	Actual platform metrics, error messages and local diagnostics; label each metric's source.
Robustness	Out-of-sample, sensitivity, subperiod, turnover and correlation observations where available.
Decision	Reject / revise / investigate / review for submission; clear reason and reviewer.
Next experiment	One specific follow-up that tests a hypothesis, not a vague request for improvement.

Appendix B. Definition of done

The MVP is done when all of the following are true:

- One manually researched baseline can be represented and reproduced from stored metadata.
- The system can store hypotheses, expressions, simulation results and decisions without overwriting history.
- Expression validation is deterministic and covered by unit tests.
- AI outputs must satisfy a strict schema and use only allowed fields/operators supplied to it.
- Every experiment records prompt/model version, configuration, code version, timestamps and results.
- Known leakage checks and data-quality checks run before candidate evaluation.
- The workflow can function without BRAIN API access through a manual export/review process.
- Any programmatic connector uses authorized documentation and account permissions.
- Batch limits, dry-run mode, error handling, audit logs and a kill switch are implemented before automation.
- The system explains why each candidate was kept, rejected or sent for further review.
- Documentation states clearly that results are experimental and earnings are not guaranteed.

A sensible first release is not the one with the most AI agents. It is the smallest version that produces traceable experiments and makes fewer bad research decisions than an unstructured workflow.

Sources and important caveats

Official sources checked while preparing this guide (22 October 2026):

[1] WorldQuant BRAIN overview
https://www.worldquant.com/brain/
Public overview of BRAIN, its research workflow and platform ecosystem.

[2] WorldQuant BRAIN Research Consultant Program
https://platform.worldquantbrain.com/consultant-program/
Program details, eligibility regions and advanced features described for consultants.

[3] WorldQuant International Quant Championship guidelines
https://www.worldquant.com/brain/iqc-guidelines/
Competition-specific rules; these are separate from general consultant-program rules.

[4] WorldQuant Learn2Quant
https://www.worldquant.com/learn2quant/
Official educational series on alpha ideas, data categories, diversification and risk.

Caveats

- Platform interfaces, available data fields, supported operators, metrics, APIs and account permissions can change. Verify current documentation in your own account before implementation.
- The guide intentionally does not specify undocumented API endpoints, bypass mechanisms or credentials workflows.
- Any sample expression idea is conceptual and must be translated into currently supported syntax and validated in BRAIN.
- Research performance on historical data does not guarantee future performance, successful submission, consultant onboarding or compensation.
- This document is educational and technical guidance, not investment advice.

Build patiently. Measure honestly. Keep the failures. Let evidence—not enthusiasm—decide which alphas survive.